COMPUTER SCIENCE + COMPUTER ENGINEERING

Module 18 Concurrency and parallel programming

Individual module / Expanded Volume 04

Four lessons with original foundations, expanded study, worked examples, laboratory checkpoints and quizzes.

Lessons 18.1–18.4 · 20 questions · One integration project

Package 05 · Expanded Volume 04 · September 28, 2026
Self-study teaching edition. Completion requires independent implementation, testing and review.

Module 18: Concurrency and parallel programming

Original prerequisites: Modules 12 and 16-17.

Integration goal: Bounded native job system. Integrate the native queue and finite task graph. Use independent output slots and explicit dependencies rather than one giant lock around useful computation.

18.1 — Races, mutexes and happens-before

Learning objective: Protect shared invariants using a clear synchronization model.

Original foundation

Concurrent execution interleaves operations. A C++ data race occurs when conflicting accesses are not properly ordered and at least one is a write, with the relevant accesses not atomic; this leads to undefined behavior. A race condition is a broader logical timing defect and can exist even when individual accesses are atomic. A mutex protects a shared invariant only when all relevant participants follow the same locking protocol. The critical section must cover the complete operation that needs to be atomic at the application level. Ownership and message passing can reduce shared mutable state. Document the happens-before relationships rather than relying on "it usually runs first."

Expanded study

Separate language races from logical races

A C++ data race concerns conflicting memory accesses without the synchronization required by the language. A logical race can remain even when each individual field access is atomic. For example, reading a sufficient balance and later debiting it are separate operations unless one protocol protects the whole decision. Testing repeatedly on one machine cannot replace the language-level ordering argument. [R11]

Protect an invariant rather than a variable name

The Inventory example has two fields: gold and stock. A successful purchase must reduce both consistently, while a rejected purchase changes neither. All reads participating in that state, including a diagnostic snapshot, follow the same mutex protocol. Returning gold and stock through two unrelated unlocked getters would permit a report that combines values from different moments. The mutex belongs with the state so callers cannot easily forget the relationship.

Locate the linearization point

Each buy call checks affordability and availability, applies the updates and releases the mutex. Relative to other callers following the protocol, the operation behaves as one indivisible purchase. The actual lock interval is small and contains no file or network I/O. A snapshot also acquires the mutex, so it observes a coherent pair. The synchronization guarantee comes from the mutex operations, not from an expectation that one thread is scheduled first. [R11, R16]

Use tests to challenge the implementation

Stress tests check counts and conservation after joining all threads. Tests should include exact funds, no stock, insufficient funds and invalid prices. A race detector can reveal certain erroneous executions, but a clean report does not establish all paths, compiler targets or progress properties. Do not run intentionally undefined non-atomic C++ races as the oracle for a correct answer; illustrate a lost update with a deterministic interleaving table instead. [P05]

Worked reasoning

Start with gold=100 and stock=20. Four threads each attempt twenty purchases at price 10. At most ten purchases can succeed; the final coherent state must be gold=0, stock=10. Which thread succeeds is unspecified and irrelevant to this invariant. By contrast, a naive check-then-subtract interleaving could let two threads both approve spending the same last ten gold. The correct test asserts the invariant, not one scheduling order.

Runnable example

File: `native/examples/lesson_18_1.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 18.1 --build-dir build
```

```
#include "lab_18_1.hpp"
#include <atomic>
#include <iostream>
#include <thread>
#include <vector>
int main() {
    course::Inventory inventory(100,20);
    std::atomic<int> sold{0};
    std::vector<std::jthread> workers;
    for (int i=0;i<4;++i) {
        workers.emplace_back([&]{
            for(int j=0;j<20;++j) {
                if(inventory.buy(10)) sold.fetch_add(1);
            }
        });
    }
    for(auto& thread:workers) thread.join();
    const auto [gold,stock]=inventory.snapshot();
    std::cout<<"sold="<<sold.load()<<" gold="<<gold<<" stock="<<stock<<'\n';
}
```

Expected output from the supplied reference in the validated environment:

```
sold=10 gold=0 stock=10
```

Practical checkpoint

Implement a shared counter and a compound inventory transaction with proper synchronization. Pass: stress test, inspect with an available race detector and explain why detector silence alone is not proof.

Expanded contract: Native Inventory(gold,stock) rejects negative initial values. buy(price) requires price>0; it changes both fields atomically under its mutex or changes neither. snapshot() returns a coherent pair. Destruction occurs only after users have joined.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_18_1.hpp`. Use `python3 check_lab.py 18.1 --student` after implementation.

Independent transfer

Both gold and stock are atomic integers. Is that enough to make a purchase across both fields transactional?

Quiz — answer before opening the key

1. **Can individually atomic reads/writes still form a logical race?**
2. **What must a mutex protocol protect?**
3. **Why is "this thread usually runs first" insufficient?**
4. **Why does snapshot() also lock?**
5. **Does the order of successful buyer threads need to match between test runs?**

Assigned study

R11: Concurrency and Parallelism; R16: Locks and Concurrency Bugs.

Expansion source map: R11, R16, P05. See the source directory for the exact documents and version notes.

18.2 — Condition variables, deadlocks and cancellation

Learning objective: Make waiting and shutdown correct under adverse timing.

Original foundation

A condition variable allows a thread to wait for a predicate to become true while coordinating with a mutex. Wait in a loop because wakeups do not by themselves prove the predicate. Notifications and state changes must follow a consistent protocol to avoid missed progress. Deadlock can arise from circular resource dependencies. A lock-order rule or reduced lock scope can prevent common cases. Shutdown deserves the same rigor as steady-state execution: wake waiters, stop accepting work, define what happens to queued jobs and join threads safely. Cancellation must specify whether work is abandoned, completed or rolled back.

Expanded study

Write the queue state machine

The bounded queue owns fixed storage, head, tail, a size count and a closed flag. Its invariant is 0 <= size <= capacity. The producer may write only when space exists and the queue is not closed. The consumer may read only when data exists. Size disambiguates empty and full when head and tail happen to be equal after wrapping. Keeping storage fixed after construction makes the queue's own capacity easy to state.

Wait on state, not on a notification

The producer waits for closed OR size<capacity. The consumer waits for closed OR size>0. The predicates are checked while holding the same mutex that protects updates. A condition-variable wait releases the mutex while sleeping and reacquires it before checking again. A notification is a hint to recheck, not ownership of an item or a guarantee that another consumer has not already taken it. [R11, R16]

Make closure a supported transition

Closing with drain=true rejects new pushes while allowing already accepted values to be popped. Once closed and empty, pop returns no value. Closing with drain=false discards values still queued; a value already popped belongs to its consumer and cannot be revoked by clearing the queue. Close notifies both producers waiting for space and consumers waiting for data. Leaving either group asleep can make shutdown hang.

Join only after releasing the resources workers need

The owner stops producers, closes the queue according to policy, lets consumers observe the transition and joins their threads. It must not hold the queue mutex while joining a consumer that needs that mutex to finish. Destruction of a queue with live waiting threads is not made safe by having a destructor. The reference deliberately requires the owner to arrange closure and joining; this responsibility belongs in the ownership map. [R11]

Worked reasoning

Capacity is two. Push 7 and 8, then close in drain mode. The next pops return 7, 8 and no value; a later push returns false. Under discard mode, closing before either pop causes the next pop to return no value immediately. In both cases, accepted-but-unprocessed work must be accounted for by the application. A queue closing successfully does not imply every accepted task completed successfully.

Runnable example

File: `native/examples/lesson_18_2.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 18.2 --build-dir build
```

```
#include "lab_18_2.hpp"
#include <iostream>
#include <thread>
#include <vector>
int main() {
    course::BoundedQueue queue(2);
    std::vector<int> seen;
    std::jthread consumer([&]{
        while(auto value=queue.pop()) seen.push_back(*value);
    });
    for(int value=1;value<=5;++value) queue.push(value);
    queue.close();
    consumer.join();
    for(auto value:seen) std::cout<<value<<' ';
    std::cout<<'\n';
}
```

Expected output from the supplied reference in the validated environment:

```
1 2 3 4 5
```

Practical checkpoint

Create a producer/consumer queue with graceful shutdown and failure propagation. Pass: test stopping with an empty queue, a full queue and active consumers; no thread remains blocked indefinitely.

Expanded contract: Native BoundedQueue(capacity) accepts 1..65536. push waits for space or closure and returns false after closure. pop waits for data or closure and returns optional<int>. close(true) drains; close(false) discards pending entries. Already popped work is not cancelled, and the owner must join users before destruction.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_18_2.hpp`. Use `python3 check_lab.py 18.2 --student` after implementation.

Independent transfer

A stop flag is set but an empty-queue consumer never wakes. What must change?

Quiz — answer before opening the key

1. **Why test a condition-variable predicate in a loop?**
2. **What does a consistent lock order prevent?**
3. **Why must shutdown notify blocked workers?**
4. **Why must closing a full queue notify producers as well as consumers?**
5. **What happens to work already removed by a consumer when close(false) runs?**

Assigned study

R16: Condition Variables, Semaphores and Concurrency Bugs; R11: locking guidance.

Expansion source map: R11, R16. See the source directory for the exact documents and version notes.

18.3 — Atomics, memory ordering and false sharing

Learning objective: Use low-level synchronization only with a complete correctness argument.

Original foundation

Atomic operations provide indivisible access according to their type and memory-order rules. Acquire/release synchronization can publish earlier writes when the specified read observes the release sequence appropriately. Relaxed ordering does not provide the same cross-object publication guarantees. Volatile is not a substitute for C++ thread synchronization. False sharing occurs when independent frequently written data occupy a shared cache line, causing coherence traffic. Padding may help a measured case but increases footprint. Lock-free algorithms also face reclamation and progress issues; an atomic pointer alone does not make referenced storage safe to free. Prefer a proven library design unless low-level implementation is the explicit learning goal.

Expanded study

Draw the complete publication chain

The one-shot example first writes a non-atomic payload, then stores true to ready with release ordering. The consumer loads ready with acquire ordering until it observes that publication, and only then reads payload. This connects the producer's earlier write to the later read through a specified synchronization relationship. Reading the initial false value does not establish that publication. [R11]

Restrict the protocol to what was proved

There is one producer, one consumer and one publication. The producer does not modify payload after setting ready. The flag is not reset and the storage remains alive until both participants finish. Reusing the same flag for many messages adds ownership and generation problems that this proof does not solve. A consumer reading an old true flag while a new payload is being written would not be following the one-shot contract.

Choose weaker ordering only for a reason

A relaxed atomic counter can count independent events when it is not being used to publish other data. That does not make a relaxed ready flag sufficient for a non-atomic payload. Similarly, volatile does not supply C++ inter-thread synchronization. Joining a worker also establishes completion relationships, but the publication example deliberately reads before the explicit join to teach the acquire/release chain. [R11]

Separate false sharing from a data race

Two independent atomic counters may be logically safe but contend when their storage shares a coherence block. The benchmark compares compact counters with a type aligned to a chosen 64 bytes. That number is an experimental layout choice, not measured cache-line information; a target with different granularity can behave differently. Thread startup and scheduling are included in the measured interval. Padding changes both sharing and memory footprint, so report the tradeoff instead of calling padding universally faster. [R15]

Worked reasoning

The required ordering is payload write -> release store -> acquire load observing true -> payload read. For input 42, the consumer returns 42. The reference creates fresh storage on every call; running it one hundred times means one hundred independent one-shot experiments, not a reusable-channel proof. The separate false-sharing experiment uses relaxed atomics for independent increments and reads final values only after joining both writers.

Runnable example

File: `native/examples/lesson_18_3.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 18.3 --build-dir build
```

```
#include "lab_18_3.hpp"
#include <iostream>
int main(){std::cout<<"published="<<course::publish_once(42)<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
published=42
```

Practical checkpoint

Write a publication litmus test with a documented happens-before diagram. Measure a padded versus unpadded counter layout. Pass: do not claim stress tests prove memory-model correctness or assume a fixed cache-line size without labeling it.

Expanded contract: Native publish_once(uint64_t) creates one publication, returns the published value and joins the producer before local storage dies. The exercise requires a correct acquire/release or stronger protocol; stress-test success alone is insufficient.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_18_3.hpp`. Use `python3 check_lab.py 18.3 --student` after implementation.

Independent transfer

A developer clears ready and writes another payload immediately after publication. Does the original proof still apply?

Quiz — answer before opening the key

1. **Is volatile a C++ inter-thread synchronization mechanism?**
2. **What is false sharing?**
3. **Why is freeing a node difficult in lock-free structures?**
4. **Why is a relaxed counter acceptable in the padding benchmark but not a drop-in ready flag replacement?**

5. **Does alignas(64) prove two counters cannot false-share on every target?**

Assigned study

R11: atomics and memory model; R16: concurrency; R37: later GPU memory-model comparison.

Expansion source map: R11, R15, R16, P05. See the source directory for the exact documents and version notes.

18.4 — Task graphs, work stealing and scaling limits

Learning objective: Parallelize independent work and measure end-to-end benefit.

Original foundation

A task graph expresses jobs and dependencies. Parallel execution is possible among ready tasks that do not conflict. Work stealing distributes queued tasks dynamically, but scheduling overhead can exceed the benefit of tiny jobs. Batch work at a granularity that matches the workload and hardware. Amdahl's law models the limit from a serial fraction: speedup = 1/(s+(1-s)/p) under its simplifying assumptions. Communication, contention and load imbalance can reduce real speedup further. A job system must handle exceptions, ownership, completion and shutdown, not only start threads. Benchmark the actual simulation rather than an empty task loop.

Expanded study

A dependency is a data-availability contract

In the graph, deps[i] lists prerequisites of task i. A task is ready only after those prerequisites have reached a terminal state; it may run only if they succeeded. The graph must be validated before starting workers: unknown indices, self-dependencies, duplicate edges and cycles are rejected. Otherwise a malformed graph can make every worker wait for a prerequisite that never completes.

Calculate work and span separately

Work is the total useful task cost in a chosen model. Span is the longest dependency path. Even with unlimited ideal workers, a computation cannot finish faster than its span. With p ideal workers it also cannot finish faster than work/p. These lower bounds exclude scheduling and communication overhead. The critical_path exercise computes the longest path using topological processing and rejects duration addition that would overflow its uint64_t model.

Read the scheduler actually supplied

The reference executor keeps a finite graph and a shared-state mutex. Workers claim ready tasks, release the mutex before calling task code, then publish done or failed status under the mutex. Dependents of a failed task become blocked rather than running with invalid inputs. Captured result storage must still have disjoint writes or a documented dependency/locking relationship. The reference is a global ready-scan scheduler, not a work-stealing implementation.

Understand the next architecture before adopting it

Work stealing gives workers local queues and permits idle workers to take suitable work elsewhere. It can reduce some contention and balance irregular jobs, but introduces queue ownership, stealing races and more difficult reclamation/progress questions. Learn the simpler executor's correctness first. Cancellation in this package is demonstrated by the bounded queue and Gate D service; the finite graph reference completes healthy work and blocks failed descendants, and does not provide arbitrary interruption of task bodies. [R11, R16]

Worked reasoning

Two independent tasks have durations 2 and 5; a one-unit aggregation depends on both. Work is 8, span is 6, and the ideal two-worker lower bound is max(8/2,6)=6. A serial chain would take 8. The executable example writes 6 and 9 into disjoint slots, then its dependent aggregation produces 15. Worker count may change the execution order of independent tasks but must not change the required aggregate.

Runnable example

File: `native/examples/lesson_18_4.cpp`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 18.4 --build-dir build
```

```
#include "task_graph.hpp"
#include <array>
#include <iostream>
int main(){std::array<int,3> output{0,0,0};
std::vector<course::Task> graph{{{},[&]{output[0]=6;}},{{},[&]{output[1]=9;}},{{0,1},[&]
{output[2]=output[0]+output[1];}}};
auto report=course::execute_graph(graph,2);
for(auto s:report.state)if(s!=course::TaskState::done)return 1;
std::cout<<"result="<<output[2]<<" modeled_span="<<course::critical_path({2,5,1},{{},{},
{0,1}})<<'\n';}
```

Expected output from the supplied reference in the validated environment:

```
result=15 modeled_span=6
```

Practical checkpoint

Implement a small task graph for independent entity updates followed by aggregation. Pass: compare 1, 2 and available worker counts, verify results and explain a case where more workers hurt.

Expanded contract: Native critical_path(durations,deps) accepts at most 1024 nodes, rejects inconsistent dimensions, invalid/duplicate edges and cycles, and returns the longest modeled path with checked uint64 arithmetic. execute_graph is a separate worked integration reference, not the function supplied by the student exercise.

Complete the four implementation/evidence steps and the additional problems in the workbook. Student interface: `native/student/lab_18_4.hpp`. Use `python3 check_lab.py 18.4 --student` after implementation.

Independent transfer

All workers execute parent jobs that synchronously wait for their own child jobs in the same bounded pool. What failure is possible?

Quiz — answer before opening the key

1. **What is the ideal maximum speedup with a 25% serial fraction?**
2. **Why can very small tasks reduce performance?**
3. **What does a task dependency express?**
4. **For a 20% serial fraction and four ideal workers, what is Amdahl speedup?**
5. **Does the supplied executor implement work stealing?**

Assigned study

R15: Parallel Processing; R16: concurrency; R11: task-oriented concurrency guidance.

Expansion source map: R11, R15, R16. See the source directory for the exact documents and version notes.

Module integration checkpoint

Original project brief: Implement a bounded job system with dependency handling, shutdown, failure propagation and stress tests. Compare one-thread and multi-thread execution on a real workload.

Integrate the native queue and finite task graph. Use independent output slots and explicit dependencies rather than one giant lock around useful computation.

1. Queue contracts for capacity, rejection, drain, discard and destruction after joined users.
2. A happens-before explanation for the compound inventory operation and one-shot publication.
3. A graph with independent jobs, an aggregation dependency and a deliberately failed predecessor.
4. Stress and shutdown checks with external timeouts; memory/concurrency diagnostics where supported.
5. An equal-work comparison using 1, 2 and a declared available worker count; keep startup/scheduling costs explicit.

Live review: A prerequisite fails while independent work remains ready. Demonstrate that dependent work is blocked, independent work finishes, and shutdown does not wait forever.

The workbook supplies the additional exercises and five-question module review. Preserve at least five new edge/failure cases beyond copied examples. A mentor reviews the reasoning as well as the outputs.